

实验报告

一、文档中的问题

1. 我之前没有参与过需要多人协作开发的项目
2. Git设计“暂存-提交”两步流程，核心原因在于：
 - 可以选择性的提交修改，也可以分批提交不同修改，给开发人员提供更灵活的版本管理方式
 - 暂存区类似于一个缓冲区，可以在正式提交之前预览、调整、撤回待提交的修改内容，提供更大的容错空间
3. git branch 和 git branch -a 的区别是什么？

- git branch：仅列出本地分支，即你在本地仓库中创建的分支。

```
feature
* main
```

- git branch -a：列出所有分支，包括本地分支和远程跟踪分支，显示远程仓库中存在但本地尚未创建的分支。

```
feature
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

二、实验

建立个人仓库

使用模板仓库创建个人仓库，然后克隆到本地；

编辑 main.c，加了一句 printf 语句，编译运行得到正确输出：

```
Hello, world!
Hi, What's your name? ^v^
```

提交一次 commit，成功。

学习分支管理

新建 feature 分支，在 feature 分支上修改 main.c 中 printf 的内容为 "Hi, what's your name? ^v^\n"

切换回main分支，修改 main.c 的 printf 为不同内容 "Hi! What's your name? ^v^\n"

合并feature到main，产生合并冲突：

```
TestLab > C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, world!\n");
7      采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
8      <<<<<< HEAD (当前更改)
9      printf("Hi! What's your name? ^v^\n");
10     =====
11     printf("Hi, what's your name? ^v^\n");
12     >>>>>> feature (传入的更改)
13 }
```

解决冲突：选择“采用当前更改”，然后完成合并。

三、阅读网页

注：选择了前两个，仅记录了我认为后续需要回顾的内容

1. Commit Message 规范

这篇文章介绍了Angular规范的Commit Message格式，每条提交信息包括Header、Body和Footer三个部分，Header必需，Body和Footer可省。

Header

1. type（必需）

- feat：新功能（feature）
- fix：修补bug
- docs：文档（documentation）
- style：格式（不影响代码运行的变动）
- refactor：重构（即不是新增功能，也不是修改bug的代码变动）
- test：增加测试
- chore：构建过程或辅助工具的变动

feat和fix类型的commit将肯定出现在Change log之中，其他建议不放

2. scope（可选）

用于说明 commit 影响的范围，比如数据层、控制层、视图层等等

3. subject（必需）

描述commit的目的

- 以动词开头，使用第一人称现在时
- 第一个字母小写
- 结尾不加句号

注：我完成实验后才阅读这篇文章，因此本次实验提交的Commit Message没有按照这个格式

Body

对commit的详细描述

- 使用第一人称现在时
- 应该说明代码变动的动机，以及与以前行为的对比

Footer

只用于两种情况

1. 不兼容变动

如果当前代码与上一个版本不兼容，则Footer部分以 `BREAKING CHANGE` 开头，后面是对变动的描述、以及变动理由和迁移方法

2. 关闭 Issue

如果当前commit针对某个issue，那么可以在Footer部分关闭这个issue

Revert

特殊情况：如果当前commit用于撤销以前的commit，则必须以 `revert:` 开头，后面跟着被撤销commit的Header

Change log

如果所有Commit都符合Angular格式，那么Change log可以用脚本自动生成

工具：[conventional-changelog](https://github.com/conventional-changelog/conventional-changelog)，运行下面的命令即可：

```
$ npm install -g conventional-changelog
$ cd my-project
$ conventional-changelog -p angular -i CHANGELOG.md -w
```

上面命令不会覆盖以前的 Change log，只会在 `CHANGELOG.md` 的头部加上自从上次发布以来的变动

如果想生成所有发布的Change log，要改为运行下面的命令：

```
$ conventional-changelog -p angular -i CHANGELOG.md -w -r 0
```

为了方便使用，可以将其写入 `package.json` 的 `scripts` 字段

```
{
  "scripts": {
    "changelog": "conventional-changelog -p angular -i CHANGELOG.md -w -r 0"
  }
}
```

以后直接运行命令 `$ npm run changelog`

2. Git Flow 分支控制

核心分支与职责

- **master**: 永久分支，与线上环境一致，保持最高稳定性，每次提交后立即打 tag
- **develop**: 永久分支，基于 master 分离，作为开发阶段的公共分支
- **feature**: 临时分支，从 develop 创建，用于开发新功能，完成后合并回 develop（使用 `--no-ff` 禁止快进合并），随后删除
- **release**: 临时分支，从 develop 创建，用于测试阶段修复 Bug，完成后合并到 master 和 develop，随后删除
- **hotfix**: 临时分支，用于紧急修复线上 bug，直接从 master 创建，修复后合并回 master 和 develop，随后删除

我对“为什么要学习 Git”的理解

结合以上两篇文章，我认为学习 Git 的重要性体现在以下几个层面：

Git 是团队协作的基础工具。现代软件开发几乎不可能由一个人独立完成，Git 通过分支管理、合并、冲突解决等机制，让多人可以在同一项目上高效协作。Git Flow 分支模型进一步规范了协作流程，使不同角色的开发者能够有序地推进各自的工作。

Git 提供了完整的版本追溯能力。通过规范的 Commit Message，团队成员可以快速了解每次改动的原因和内容，方便排查问题和回退版本。Git 可以让开发者将文件回溯到之前的状态，查看是谁、何时、为什么修改了某处代码。

Git 是行业通用技能。Git 是目前最主流的版本控制系统，大量软件项目依赖它进行版本控制。掌握 Git 不仅是当前学习的需要，也是未来职业发展的必备技能。